



Code Trends, Quantified: Mapping the Programming Language Ecosystem

BA820 – Team 14

Arshdeep · Ahrar · Vishesh · Drishti

Why Quantify Code Trends?

Programming languages evolve rapidly, yet online popularity rarely mirrors real-world hiring demand. We set out to **quantify adoption patterns using data** rather than intuition.

❏ **Key Idea:** Find natural groupings of languages without predefined categories - letting structure emerge from the data itself.

Stakeholders

- Students choosing which language to learn
- Companies selecting technology stacks
- Recruiters tracking market demand

Why it matters

Misleading popularity signals can lead to poor technology decisions.



Project Pipeline

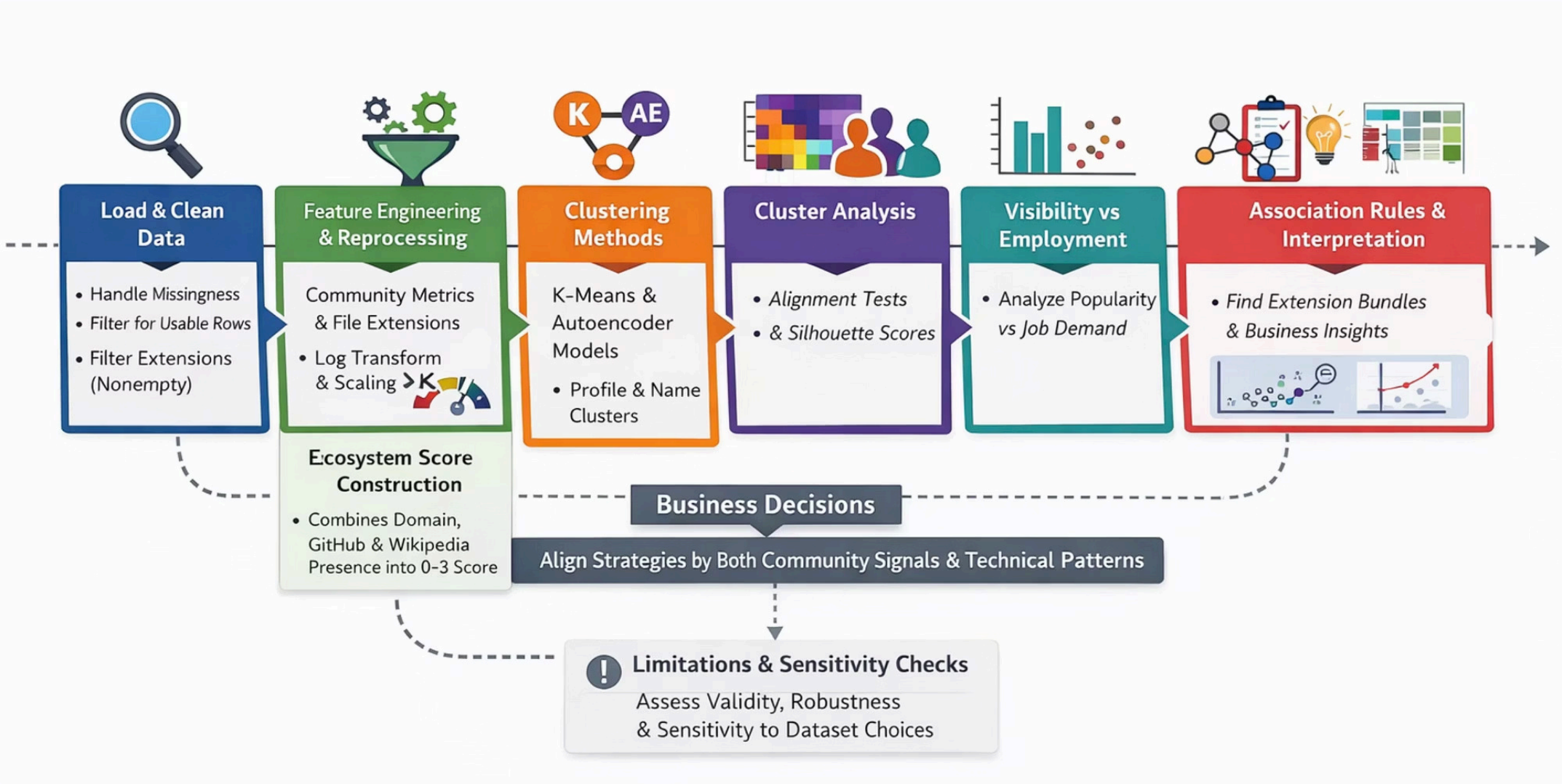


Figure 1: Pipeline for our Final Project

Dataset & Preprocessing

Dataset: languages.csv — programming languages with *community signals* (users, jobs, GitHub stars, Wikipedia views) and *technical signals* (file extensions), plus metadata like appeared and last_activity.

Key Issues:

1. **Log transform:** used $\log_{10}(x+1)$ on community metrics to reduce skew and heavy missingness.
2. **Numeric casting + cleaning:** coerced key metrics to numeric; set invalid values (e.g., negative wiki views) to NA.
3. **Extension parsing:** cleaned extensions column into an extensions list per language for consistent tokens.

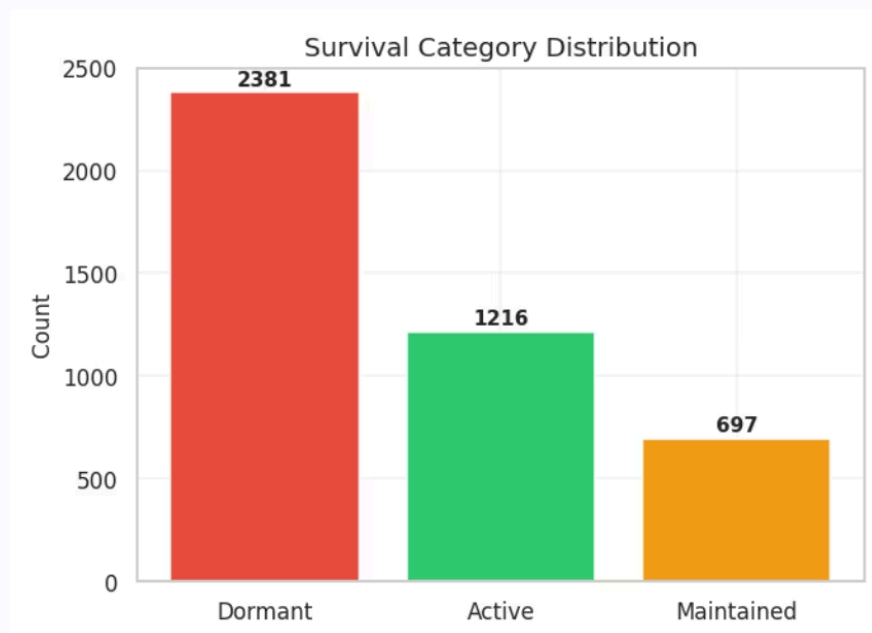


Figure 2: How relevant are the languages

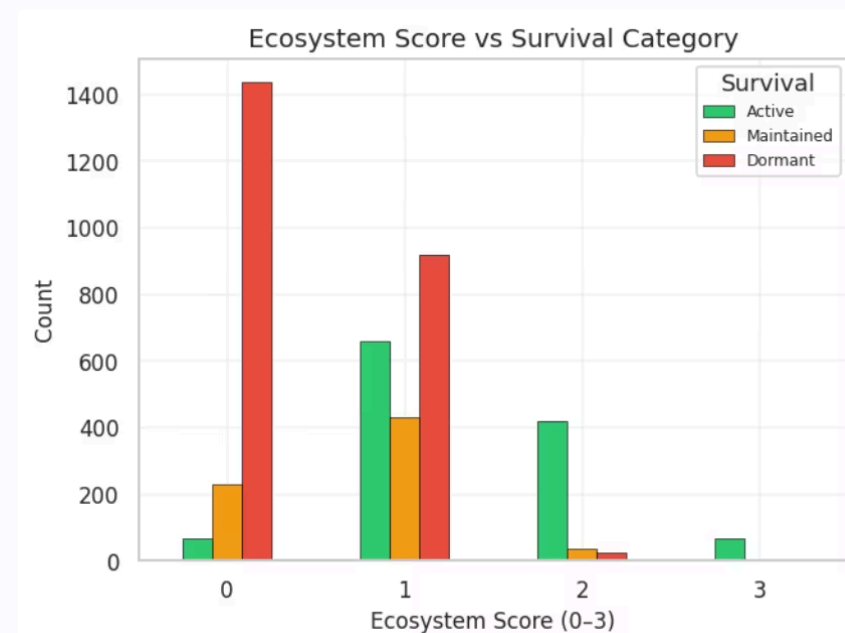


Figure 3: How many Features does languages have

Take Away: We cleaned and scaled highly skewed, incomplete data; results are meaningful but must be interpreted within the community/extension subsets caused by missingness.

K-Means Clustering

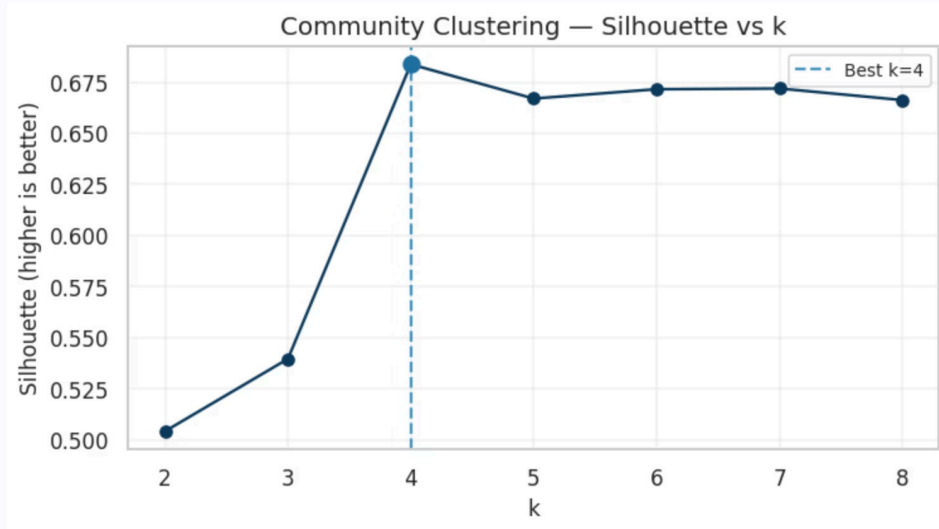


Figure 4: K=4 is optimal K for Community Clustering

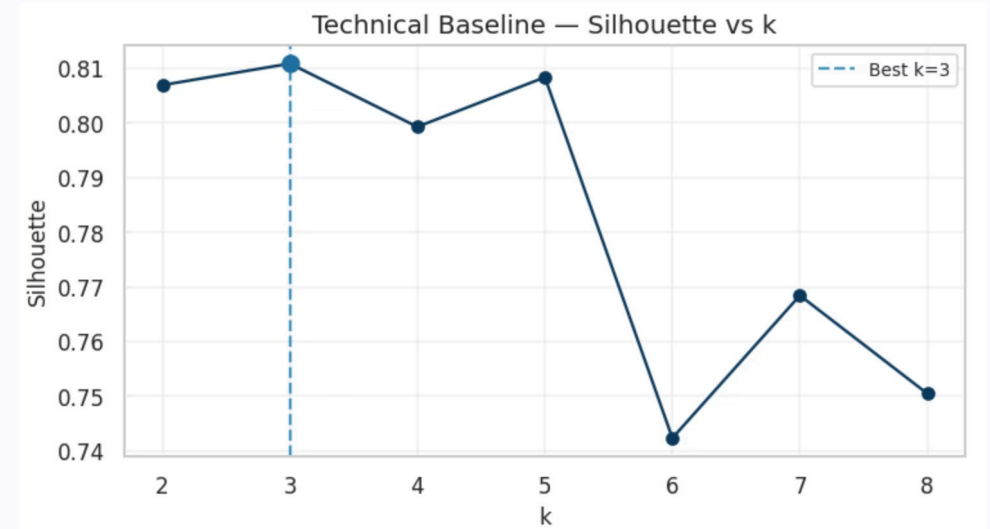


Figure 5: K=3 is optimal K for Technical Clustering



Why K-Means?

Scalable, Robust and Simple for a quick analysis.

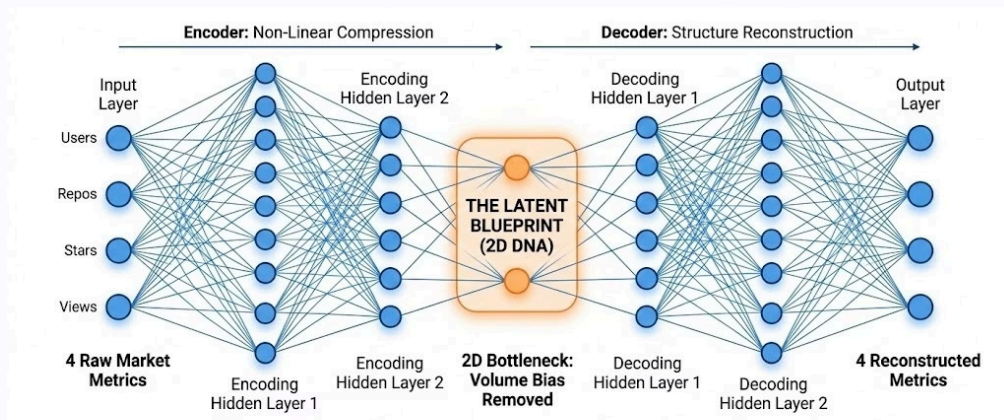


Silhouette Score

Confirm cluster separation quality, and high silhouette for both technical and community clusters indicates high internal cohesion and good separation from the neighbouring clusters

Result: Languages naturally segment into distinct structural categories, and using BoW on the extensions for each language we can see that the file extension similarity does not replicate into adoption similarity

Autoencoders: Learning Hidden Structure



Traditional models (like K-Means) were blinded by "**Volume Bias**." Python and Java are so much larger than everything else that they acted as mathematical black holes, **hiding smaller emerging trends**.

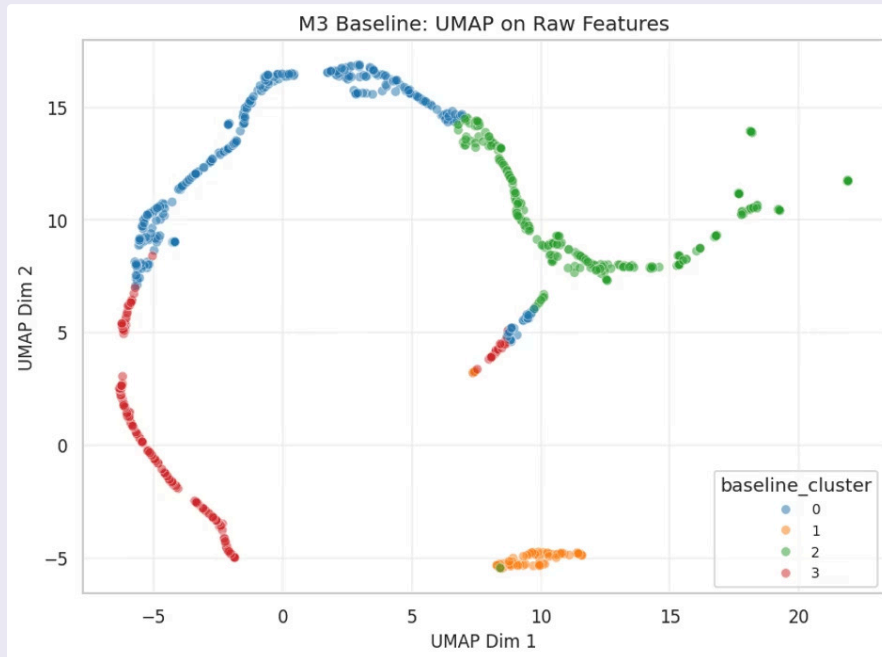
The Autoencoder uses a **non-linear bottleneck layer**. It compresses the network to ignore the *size* of the language (noise) and instead learn the underlying *relational structure* (the DNA).

How it works?

- By forcing "Active Market" data through a non-linear bottleneck, we strip away hype to reveal a technology's true market blueprint.
- Optimized a **Mean Squared Error (MSE) loss to 0.08**, ensuring the "blueprint" it learned was highly accurate.

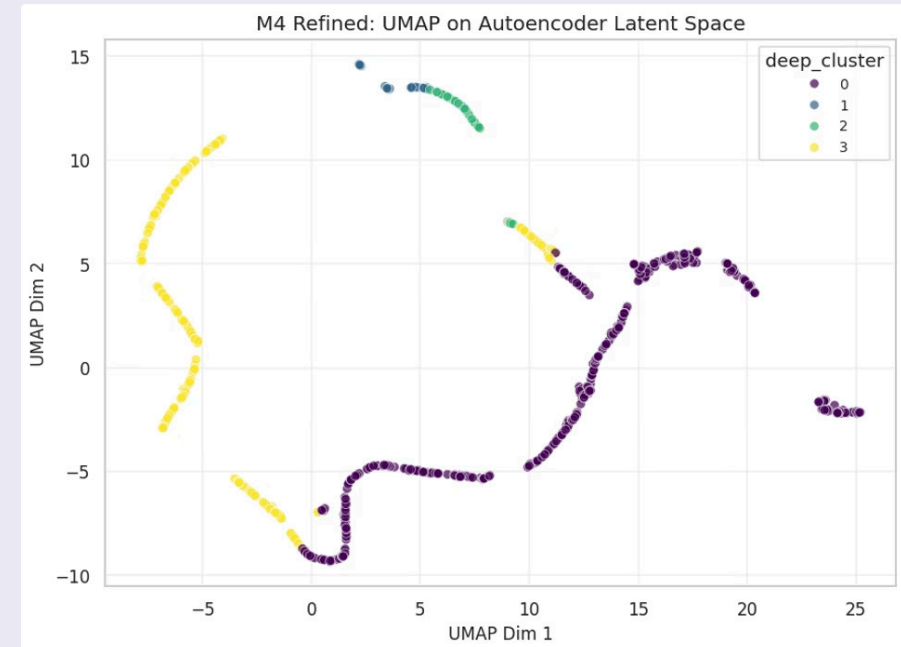
Autoencoder Insights- 62% Archetype Shift

M3: Surface Volume Bias



- Clusters are determined by raw scale (Big vs. Small).
- Significant overlap between "Famous" and "Useful" languages.
- High mathematical noise makes boundaries blurry.

M4: Latent Structural DNA



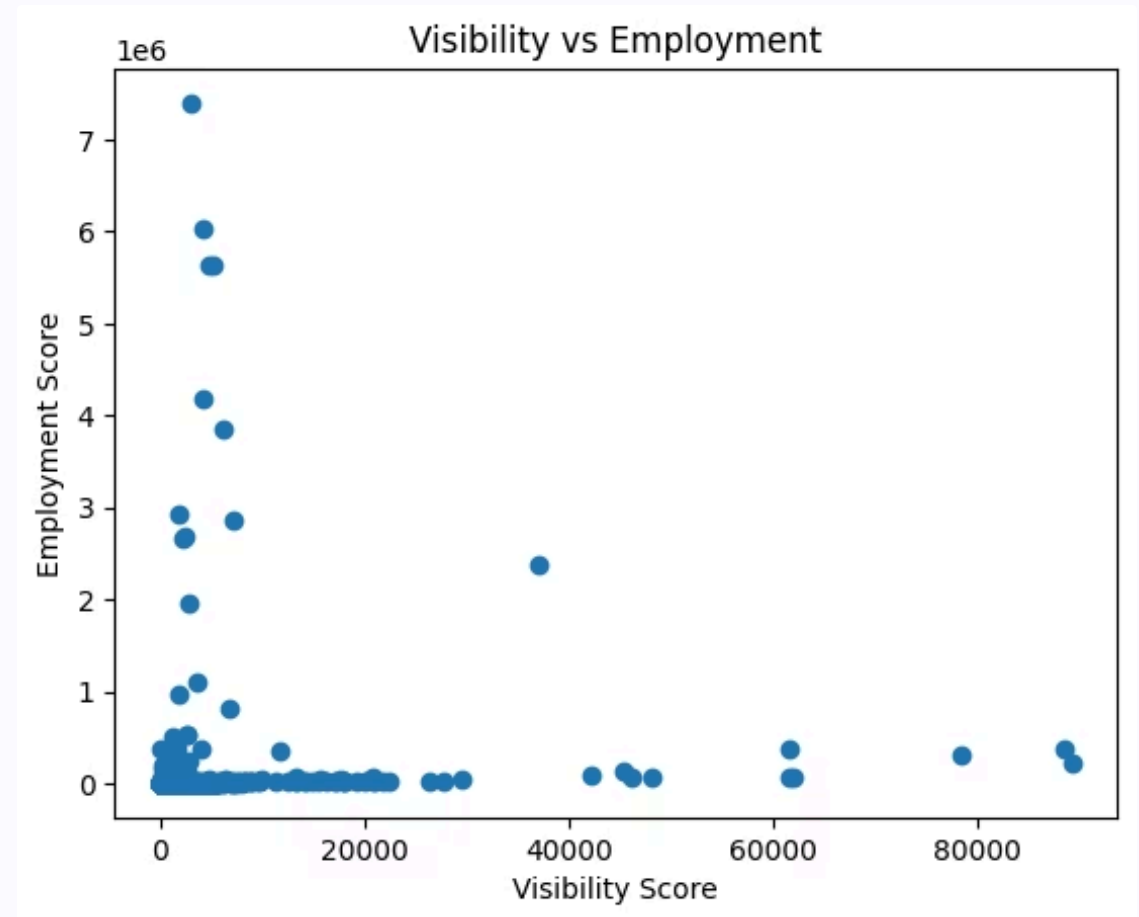
- Deep separation of functional roles (The "Hidden Titans").
- **62% of languages reclassified** into more accurate archetypes.
- Reveals distinct market "ecosystems" invisible to linear models.

Takeaway: The latent space reveals that 3 out of 5 languages were fundamentally misunderstood when looking only at surface-level popularity.

Quantifying Visibility vs. Employment

We engineered two composite features to separate **hype from hiring reality**:

Log transformation reduced skewness, and the **visibility gap** ($\log \text{visibility} - \log \text{employment}$) quantifies the mismatch between popularity and demand.



Visibility Score

GitHub stars + Wikipedia page views

Employment Score

Job postings + active user counts

Ghost Language Detection & Validation

Applying K-Means with $k = 4$ and rigorous validation uncovered a striking phenomenon: some languages are highly visible yet have negligible job demand — "**Ghost Languages.**"



Log Transformation

Applied **log transformation** to reduce skewness



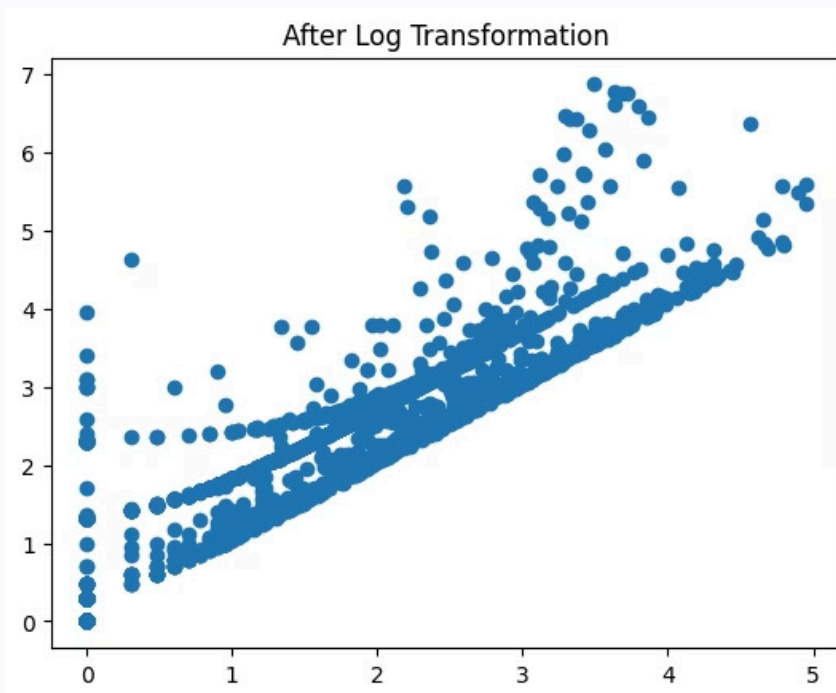
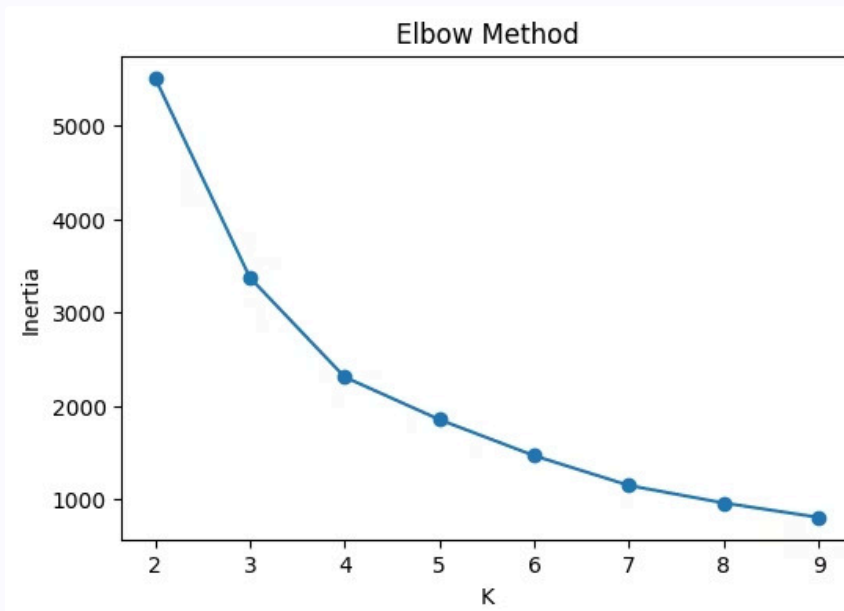
Elbow & Silhouette

Confirmed $k = 4$ as optimal cluster count



PCA Visualisation

2-D projection reveals clear cluster boundaries

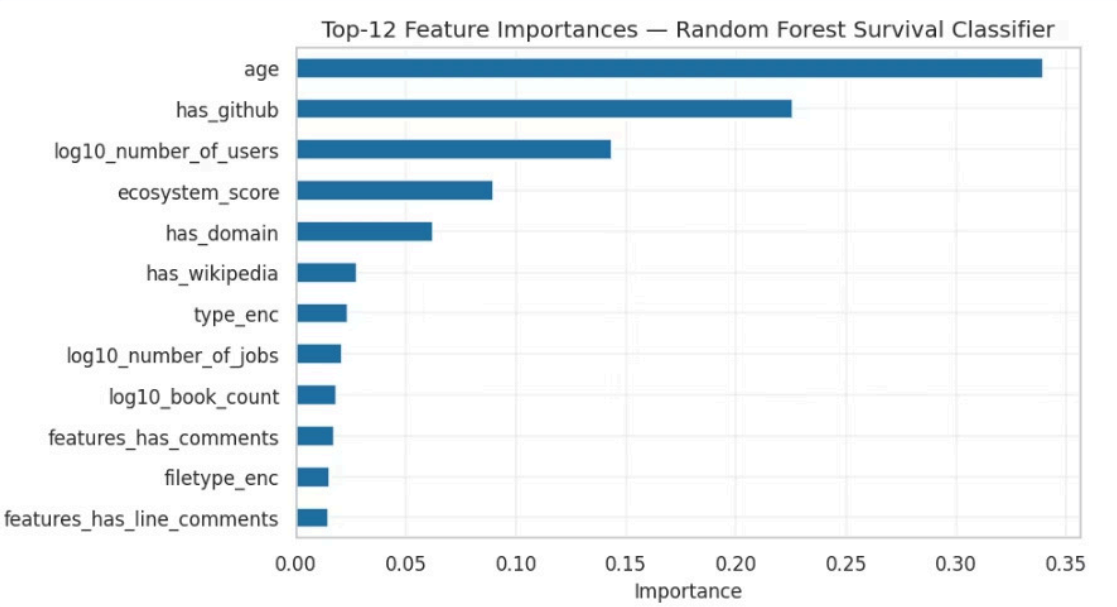


Key Predictors & Association Rules

Random Forest **Classification** to predict whether a programming language will stay Active, go Dormant, or become Maintained?

Ran **sensitivity analysis** across three different survival threshold definitions—strict, conservative, and lenient—and accuracy only varied by 1.5%. Confirms findings are robust.

Using **Market Basket Analysis**, we identified which programming languages are frequently adopted together, revealing hidden ecosystem dependencies.



	antecedents	consequents	support	confidence	lift
35	(features_has_line_comments)	(surv_Active, features_has_comments)	0.068002	0.564797	7.196552
34	(surv_Active, features_has_comments)	(features_has_line_comments)	0.068002	0.866469	7.196552
26	(features_has_line_comments, has_wikipedia)	(features_has_comments)	0.070098	1.000000	6.959481
31	(features_has_comments)	(features_has_line_comments, has_wikipedia)	0.070098	0.487844	6.959481
14	(features_has_line_comments)	(features_has_comments)	0.120168	0.998066	6.946020

Business Insights & Conclusion

Popularity $\neq$ Demand

Check ecosystem score, not stars

Ghost Languages

High visibility $\neq$ job opportunities

Hidden Archetypes Matter

30% of languages misclassified by surface metrics

Technical $\neq$ Community Adoption

Core tech wins, similar tech doesn't predict similar demand

Final Takeaway: Machine learning turned language adoption from guesswork into a predictive decision support system.

Thank You

Arshdeep · Ahrar · Vishesh · Drishti ·

BA820 – UML Team 14

Questions, feedback, and discussion welcome.

